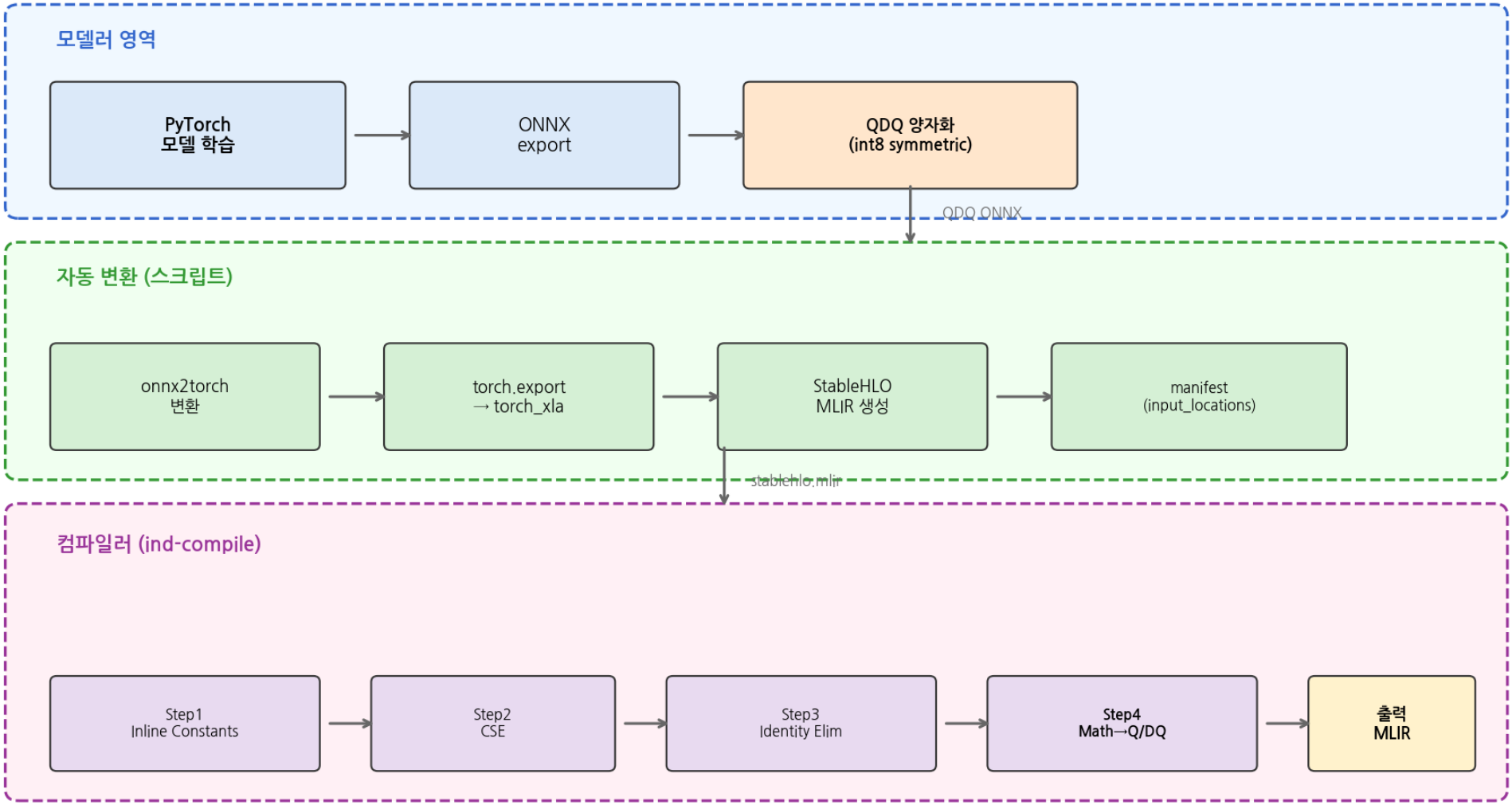
Infetron-V2 NPU
모델 조작 가이드

모델러를 위한 컴파일 가능 모델 개발 가이드

2026-03-27

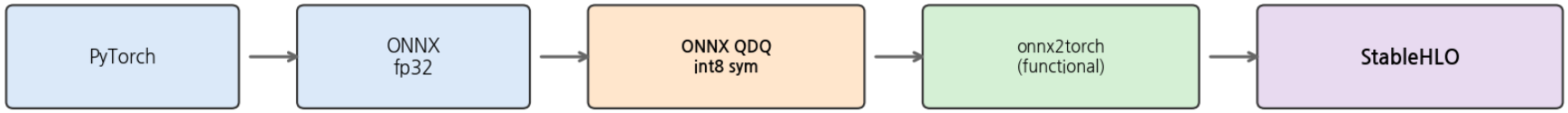
Infetron-V2 AI Compiler Team

전체 파이프라인 흐름

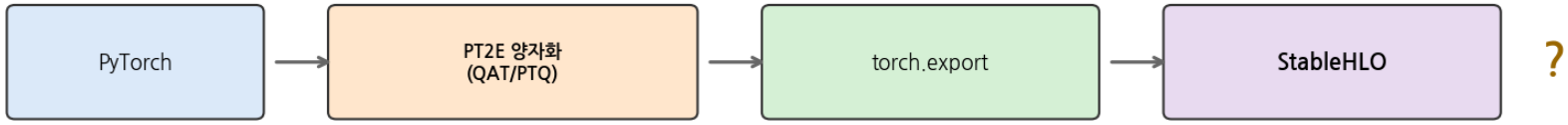

모델러 영역
PyTorch
모델 학습
ONNX
export
QDQ 양자화
(int8 symmetric)
QDQ ONNX
자동 변환 (스크립트)
onnx2torch
변환
torch.export
→ torch_xla
StableHLO
MLIR 생성
manifest
(input_locations)
stablehlo.mlir
컴파일러 (ind-compile)
Step1
Inline Constants
Step2
CSE
Step3
Identity Elim
Step4
Math→Q/DQ
출력
MLIR

두 가지 변환 경로

경로 A: ONNX 경유 (검증 완료)



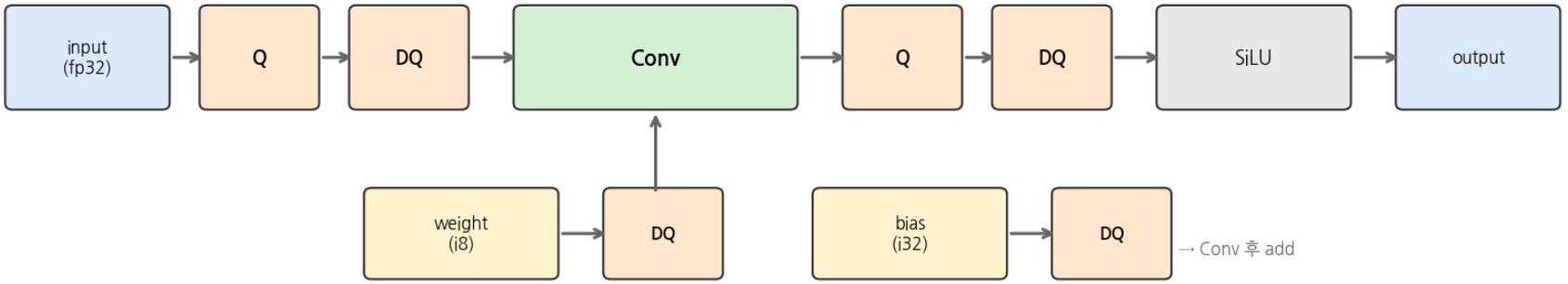
경로 B: PyTorch 직접 양자화 (미검증)



(ONNX 왕복 불필요, 향후 검증 예정)

fp32 원본 모델

QDQ 양자화 후



Q = QuantizeLinear: round(x / scale), clamp(-128, 127)

DQ = DequantizeLinear: x * scale

모델 개발 요구사항

항목	요구사항	구분
ONNX opset	17 (최소 13)	필수
Shape	정적(static) — 모든 shape 고정	필수
Batch size	1 고정	필수
양자화 형식	QDQ (QuantizeLinear + DequantizeLinear)	필수
양자화 타입	int8 symmetric (per-tensor)	필수
Conv+BN	fusion 후 export	필수
Calibration	MinMax, 20~100개 데이터	권장
전처리	모델 외부에서 수행	권장

미지원 사항

미지원 항목	대안
Dynamic shape	고정 shape으로 export
per-channel quantization	per_channel=False 사용
Mixed precision	전체 int8 또는 전체 fp32
QOperator (ConvInteger)	QDQ 형식 사용
int4 / fp16	int8만 지원
Custom ONNX op	표준 op으로 분해

1. 전체 흐름

모델을 NPU 컴파일러에 전달하는 경로는 2가지입니다.

[경로 A: ONNX 경유 — 검증 완료]

PyTorch 학습 → ONNX export → ONNX QDQ 양자화 → onnx2torch → torch_xla → StableHLO

[경로 B: PyTorch 직접 — 미검증]

PyTorch 학습 → PT2E QAT/PTQ 양자화 → torch.export → torch_xla → StableHLO

경로 A는 검증 완료된 방법이며, 경로 B는 향후 검증 예정입니다.

※ PPT 슬라이드 1~2에 다이어그램이 있습니다.

2. 모델 개발 요구사항

2.1 필수 요구사항

항목	요구사항	비고
ONNX opset	17 (최소 13)	QDQ op 지원
Shape	정적(static)	dynamic 미지원
Batch size	1	고정
양자화 형식	QDQ	QOperator 미지원
양자화 타입	int8 symmetric	per-tensor만
Conv+BN	fusion 후 export	BN 별도 불가

2.2 미지원 사항

항목	상태	대안
Dynamic shape	미지원	고정 shape export
per-channel quant	미지원	per_channel=False
Mixed precision	미지원	전체 int8/fp32
int4 / fp16	미지원	int8만 지원
Custom ONNX op	미지원	표준 op 분해

3. 변환 실행 (경로 A)

Step 0a: QDQ 양자화

```
python onnx_prepare.py --onnx model_fp32.onnx --output-dir ./work
```

출력: model_qdq_int8sym.onnx, conv_metadata.json

Step 0b: StableHLO 생성

```
python model_to_stablehlo.py \
  --model-dir ./work --onnx model_fp32.onnx --output-dir ./work
```

출력: stablehlo.mlir, weights/manifest.json

Step 1~4: 컴파일

```
ind-compile stablehlo.mlir \
  --manifest=weights/manifest.json \
  --compile-to=QDQFusion \
  -o compiled.mlir
```

디버깅 모드

```
ind-compile stablehlo.mlir \
  --manifest=weights/manifest.json \
  --compile-to=QDQFusion \
  --dump-after-each --dump-dir=./dumps \
  --print-stats --elide=32 \
  -o compiled.mlir
```

4. 스크립트 및 도구

파일	용도	위치
onnx_prepare.py	QDQ 양자화 + converter	scripts/
model_to_stablehlo.py	StableHLO + manifest	scripts/
ind-opt	개별 pass 실행	build_ind/tools/
ind-compile	전체 파이프라인	build_ind/tools/

5. 검증 방법

변환 후 다음을 확인합니다:

- Conv 수: ONNX의 Conv 수와 StableHLO의 stablehlo.convolution 수가 일치
- Weight shape: ONNX initializer shape과 manifest shape 일치
- Scale 값: QDQ scale이 StableHLO constant로 보존
- 추론 결과: 양자화 전후 accuracy 확인

검증 명령

```
# Conv 수
grep -c "stablehlo.convolution" step4.mlir

# Q/DQ 수
grep -c "uniform_quantize" step4.mlir
grep -c "uniform_dequantize" step4.mlir
```

6. 자주 발생하는 문제

문제	원인	해결
onnx2torch 실패	미지원 op	converter 추가 요청
torch.export 실패	dynamic shape	고정 shape export
Conv 수 불일치	BN 미fusion	Conv+BN fusion
Scale 오차	부동소수점	허용범위 (<1e-6)
opset 에러	opset < 13	opset=17 사용